

Architecture.md v1.1

Chapter 12 — Plugin Architecture & Agent Registration System

This chapter defines how AI OS discovers, registers, validates, loads, updates, and manages Worker Agents and future plugins.

The objective is to allow AI OS to grow by **adding new capabilities without modifying the core architecture**.

12.1 Purpose

The Plugin System shall:

- Discover Worker Agents automatically.
 - Register new agents.
 - Validate compatibility.
 - Enable or disable agents.
 - Support future third-party extensions.
 - Prevent incompatible modules from loading.
-

12.2 Design Philosophy

Every Worker Agent is treated as a plugin.

The AI OS Core should never need modification when a new Worker Agent is added.

Adding a new capability should require:

1. Create the agent.
2. Register the agent.
3. Enable the agent.

Nothing more.

12.3 Plugin Components

Each plugin consists of:

- Metadata
- Configuration
- Execution Logic
- Permission Declaration
- Health Check
- Documentation

Optional:

- Tests
- Assets
- Templates

12.4 Plugin Manifest

Every plugin shall expose a manifest.

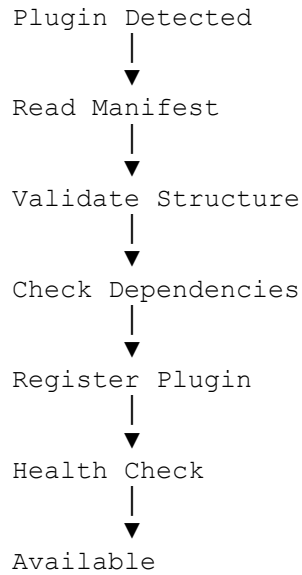
Required fields:

- Plugin Name
- Unique ID
- Version
- Description
- Author
- Supported Tasks
- Required Permissions
- Dependencies
- Entry Point

Example:

```
plugin_id: analytics_agent
version: 1.0.0
author: AI OS
entry_point: agent.py
```

12.5 Registration Workflow



If validation fails, the plugin is not loaded.

12.6 Registration Requirements

Before registration, AI OS verifies:

- Valid manifest
- Compatible version
- Required files exist
- Required permissions declared
- Entry point available
- No duplicate Plugin ID

Failure in any validation step prevents registration.

12.7 Plugin States

Every plugin exists in one state.

States:

- DISCOVERED
- REGISTERED
- ENABLED

- DISABLED
- FAILED
- UPDATING
- REMOVED

Only the Plugin Manager may change plugin states.

12.8 Plugin Manager

The Plugin Manager is responsible for:

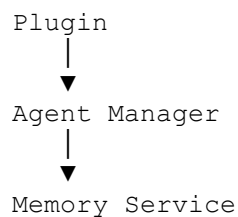
- Discovery
- Registration
- Loading
- Unloading
- Version checking
- Compatibility validation
- Health monitoring

The Plugin Manager does not execute plugin business logic.

12.9 Dependency Rules

Plugins should avoid dependencies on other plugins.

Preferred dependency:



Direct Plugin → Plugin communication is prohibited.

12.10 Version Compatibility

Each plugin declares:

- Minimum AI OS Version
- Maximum Supported Version

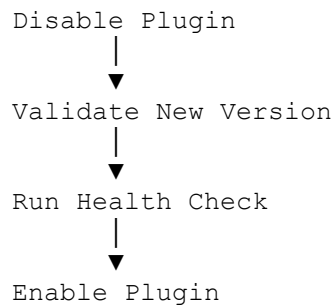
Example:

```
AI OS >= 1.1.0
AI OS < 2.0.0
```

Incompatible plugins are rejected during startup.

12.11 Plugin Updates

Updating a plugin follows this process:



Failed updates automatically restore the previous working version.

12.12 Health Checks

Every plugin implements:

- Startup Check
- Runtime Check
- Dependency Check

Possible results:

- HEALTHY
- DEGRADED
- FAILED

Failed plugins are isolated from the system.

12.13 Plugin Configuration

Each plugin may define its own configuration section.

Examples:

Analytics Plugin:

- Default report interval
- Cache size

Browser Plugin:

- Timeout
- User agent

Global configuration overrides plugin defaults when conflicts occur.

12.14 Security Requirements

Plugins must:

- Declare permissions
- Pass security validation
- Respect Human Approval rules
- Use Memory Service
- Follow Communication Contracts

Plugins may not:

- Access databases directly